

TASK 29 – E-Commerce Data Warehouse Assignment

Name: Shereen Mohamed

1. Data Generation & Ingestion Layer

- **Python API Event Generation:** Simulated a real-time e-commerce transactional pipeline generating structured JSON payloads containing order metadata, line items, payment details, shipment info, and financial summaries.
- **S3 Raw Zone Storage:** Landed the raw JSON event streams into an Amazon S3 bucket designated for unstructured/raw transactional data.

2. Data Processing & Transformation (ETL / Medallion Architecture)

- **PySpark Processing:** Processed the raw datasets using Apache Spark to clean, transform, and structure the data into a rigorous dimensional model (Star Schema).
- **Dimensional & Fact Modeling:**
 - *Dimensions created:* dim_customer, dim_product, dim_order_status, dim_payment_method, dim_shipper, dim_date, etc.
 - *Facts created:* fact_customer_sales, fact_order, fact_order_detail, fact_product_sales, fact_payment, fact_shipment.
- **Parquet Optimization:** Saved the processed tables into optimized, columnar Apache Parquet format within the Amazon S3 processed zone to ensure high performance and low storage costs.

3. Metadata Cataloging & Schema Discovery

- **AWS Glue Crawlers:** Configured and executed AWS Glue Crawlers to scan the S3 Parquet files, automatically infer schemas, partition structures, and populate the AWS Glue Data Catalog (ecommerce_curated_db).

4. Serverless SQL Analytics & Reporting (Data Lakehouse)

- **Amazon Athena Integration:** Connected Amazon Athena to the Glue Data Catalog and configured the S3 query results location.

- **Query Execution & Validation:** Executed descriptive, diagnostic, and advanced predictive SQL queries (Tasks 29–41) including:
 - Overall business KPIs (Total sales, orders, customers, and estimated profit).
 - Monthly aggregation trends and performance metrics.
 - Product and customer segmentation (Top 10 analysis, customer ranking by spending).
 - Operational analysis (Order statuses, payment gateway metrics, and shipment delivery lead times).
 - Advanced SQL & Window Functions (Cumulative running sales, Month-over-Month growth percentage using `LAG()`, and product ranking within categories using `DENSE_RANK()`).

```
C:\Windows\System32\cmd.exe - pip install fastapi uvicorn pydantic boto3 urllib3

D:\Desktop\DE projects\ecommerce-data-pipeline>mkdir docs

D:\Desktop\DE projects\ecommerce-data-pipeline>pip install fastapi uvicorn pydantic boto3 urllib3
Collecting fastapi
  Downloading fastapi-0.141.1-py3-none-any.whl.metadata (27 kB)
Collecting uvicorn
  Downloading uvicorn-0.53.0-py3-none-any.whl.metadata (6.6 kB)
Collecting pydantic
  Downloading pydantic-2.13.5-py3-none-any.whl.metadata (110 kB)
Collecting boto3
  Downloading boto3-1.43.99-py3-none-any.whl.metadata (6.6 kB)
Requirement already satisfied: urllib3 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (2.6.3)
Collecting starlette>=0.46.0 (from fastapi)
  Downloading starlette-1.6.0-py3-none-any.whl.metadata (6.4 kB)
Requirement already satisfied: typing-extensions>=4.8.0 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from fastapi) (4.15.0)
Collecting typing-inspection>=0.4.2 (from fastapi)
  Downloading typing_inspection-0.4.4-py3-none-any.whl.metadata (2.6 kB)
Collecting annotated-doc>=0.0.2 (from fastapi)
  Downloading annotated_doc-0.0.5-py3-none-any.whl.metadata (6.5 kB)
Requirement already satisfied: click>=7.0 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from uvicorn) (8.3.2)
Collecting h11>=0.8 (from uvicorn)
  Downloading h11-0.16.0-py3-none-any.whl.metadata (8.3 kB)
Collecting annotated-types>=0.6.0 (from pydantic)
  Downloading annotated_types-0.8.0-py3-none-any.whl.metadata (15 kB)
Collecting pydantic-core==2.46.5 (from pydantic)
  Downloading pydantic_core-2.46.5-cp310-cp310-win_amd64.whl.metadata (6.7 kB)
```

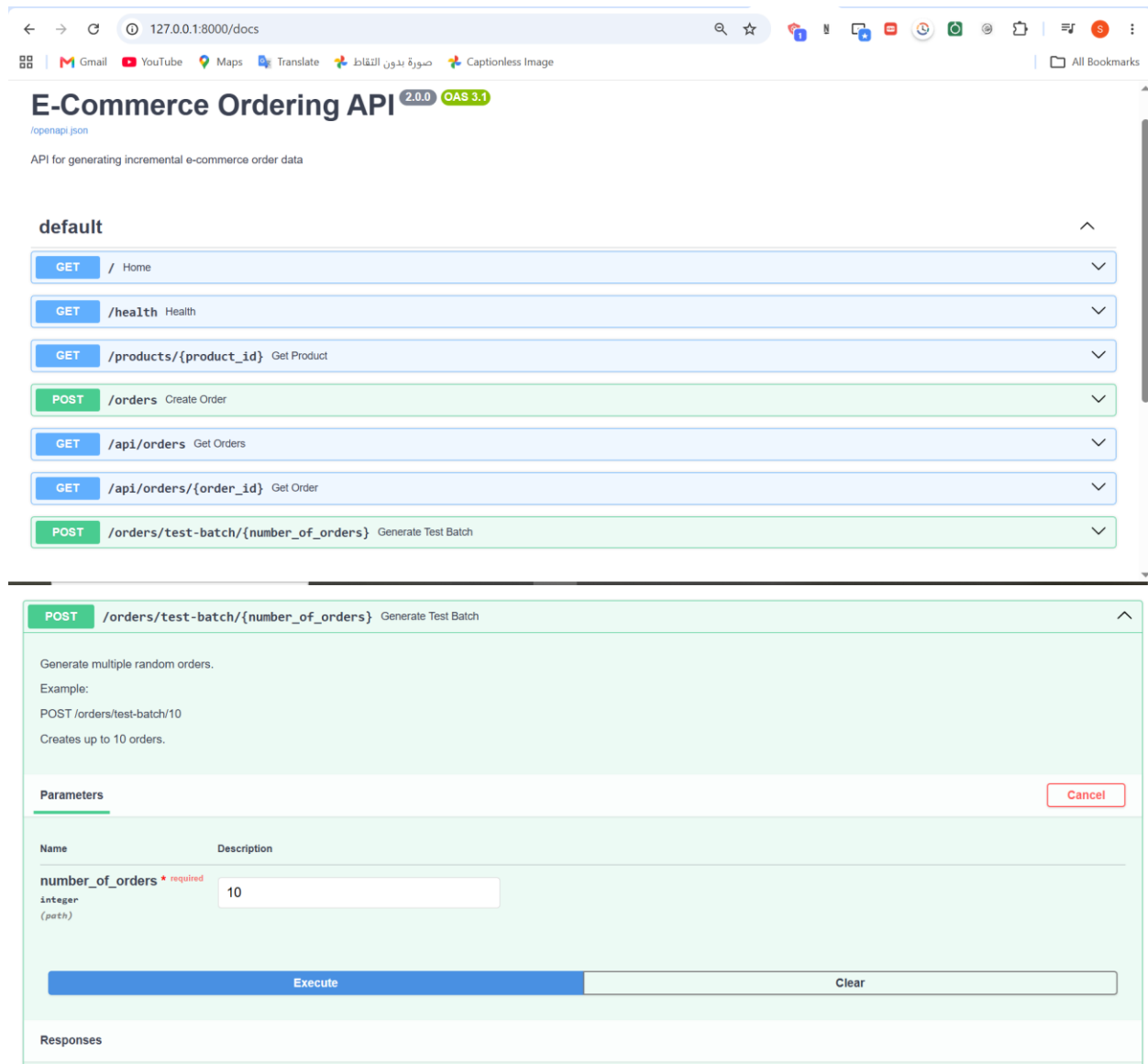
```
C:\Windows\System32\cmd.exe

thon-dateutil<3.0.0,>=2.1->botocore<1.44.0,>=1.43.99->boto3) (1.17.0)
Requirement already satisfied: colorama in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from click>=7.0->uvicorn) (0.4.6)
Collecting anyio<5,>=3.6.2 (from starlette>=0.46.0->fastapi)
  Downloading anyio-4.15.1-py3-none-any.whl.metadata (4.7 kB)
Collecting exceptiongroup>=1.0.2 (from anyio<5,>=3.6.2->starlette>=0.46.0->fastapi)
  Downloading exceptiongroup-1.3.1-py3-none-any.whl.metadata (6.7 kB)
Requirement already satisfied: idna>=2.8 in c:\users\hp\appdata\local\programs\python\python310\lib\site-packages (from anyio<5,>=3.6.2->starlette>=0.46.0->fastapi) (3.11)
Downloading fastapi-0.141.1-py3-none-any.whl (131 kB)
Downloading pydantic-2.13.5-py3-none-any.whl (472 kB)
Downloading pydantic_core-2.46.5-cp310-cp310-win_amd64.whl (2.0 MB)
----- 2.0/2.0 MB 2.0 MB/s 0:00:00
Downloading boto3-1.43.99-py3-none-any.whl (140 kB)
Downloading botocore-1.43.99-py3-none-any.whl (15.8 MB)
----- 15.8/15.8 MB 2.2 MB/s 0:00:07
Downloading s3transfer-0.19.2-py3-none-any.whl (90 kB)
Downloading starlette-1.6.0-py3-none-any.whl (75 kB)
Downloading anyio-4.15.1-py3-none-any.whl (132 kB)
Downloading exceptiongroup-1.3.1-py3-none-any.whl (16 kB)
Downloading typing_inspection-0.4.4-py3-none-any.whl (14 kB)
Installing collected packages: typing-inspection, pydantic-core, exceptiongroup, pydantic, botocore, anyio, starlette, s3transfer, fastapi, boto3
Successfully installed anyio-4.15.1 boto3-1.43.99 botocore-1.43.99 exceptiongroup-1.3.1 fastapi-0.141.1 pydantic-2.13.5 pydantic-core-2.46.5 s3transfer-0.19.2 starlette-1.6.0 typing-inspection-0.4.4

[notice] A new release of pip is available: 25.3 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

D:\Desktop\DE projects\ecommerce-data-pipeline>
```

```
D:\Desktop\DE projects\ecommerce-data-pipeline>uvicorn api.ordering_api:app --reload
[32mINFO[0m: Will watch for changes in these directories: ['D:\Desktop\DE projects\ecommerce-data-pipeline']
[32mINFO[0m: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
[32mINFO[0m: Started reloader process [36m-[118724[0m using [36m-[1mStatReload[0m
[32mINFO[0m: Started server process [36m16256[0m
[32mINFO[0m: Waiting for application startup.
[32mINFO[0m: Application startup complete.
```



Server response

Code Details

200

Response body

```
{
  "message": "Batch generation completed",
  "requested": 10,
  "created": 10,
  "orders": [
    {
      "order_id": 2000001,
      "status": "Confirmed",
      "total_amount": 4631.64
    },
    {
      "order_id": 2000002,
      "status": "Confirmed",
      "total_amount": 2751.28
    },
    {
      "order_id": 2000003,
      "status": "Confirmed",
      "total_amount": 1610.83
    },
    {
      "order_id": 2000004,
      "status": "Confirmed",
      "total_amount": 5811.79
    },
    {
      "order_id": 2000005,
      "status": "Confirmed",
      "total_amount": 1610.83
    }
  ]
}
```

Response headers

```
content-length: 726
content-type: application/json
date: Tue, 22 Sep 2026 15:06:55 GMT
server: uvicorn
```

aws [Icons] Search [Alt+S]

Amazon S3 > Buckets > aws-ecommerce-data-shereen-2026

✓ Successfully created bucket "aws-ecommerce-data-shereen-2026"

aws-ecommerce-data-shereen-2026 [Info](#)

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems

Objects (0)



Copy S3 URI

Copy UR

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your b

Find objects by prefix

	Name	Type	Last modified
--	------	------	---------------

No objects

You don't have any objects

Upload

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

D:\Desktop\DE projects\ecommerce-data-pipeline>ngrok config add-authtoken 3JgnoKGNeHnkWFgdRJYf1g8NLOC_6yu48RSppjoy1jff9C613
Authtoken saved to configuration file: C:\Users\HP\AppData\Local\ngrok\ngrok.yml

D:\Desktop\DE projects\ecommerce-data-pipeline>
```

```
C:\Windows\System32\cmd.exe - ngrok http 8000

ngrok

Dev is on ngrok.ai

Session Status      online
Account             shereenmohamed632006@gmail.com (Plan: Free)
Version             3.39.11
Region              Europe (eu)
Web Interface       http://127.0.0.1:4040
Forwarding           https://flashback-ahead-nephew.ngrok-free.dev -> http://localhost:8000

Connections          ttl      opn      rt1      rt5      p50      p90
                   0         0        0.00     0.00     0.00     0.00
```

Lambda > Functions > ecommerce-ingestion-lambda

ecommerce-ingestion-lambda

Throttle

Copy ARN

Actions

Export to Infrastructure Composer

Download

Function ARN

arn:aws:lambda:us-east-1:544733633889:function:ecommerce-ingestion-lambda

Description

-

Last modified

43 seconds ago

Function overview

Diagram

Template

+ Add trigger

+ Add destination

Code

Test

Monitor

Configuration

Aliases

Versions

Code source

Open in Visual Studio Code

Update

EXPLORER

ecommerce-ingestion-lambda

lambda_function.py

386 def lambda_handler(event, context):

580 except Exception as e:

Lambda > Functions > ecommerce-ingestion-lambda

EXPLORER

ecommerce-ingestion-lambda

lambda_function.py

386 def lambda_handler(event, context):

580 except Exception as e:

597

598

599

600

601

602

603

604

605

606

607

608

609

610

611

612

613

614

615

616

617

return {

"statusCode": 500,

"body": json.dumps({

"message":

"Failed to process orders",

"error":

str(e),

"error_type":

type(e).__name__

})

}

DEPLOY

Deploy (Ctrl+Shift+U)

Test (Ctrl+Shift+I)

TEST EVENTS [NONE SELECTED]

+ Create new test event

ENVIRONMENT VARIABLES

Lambda Deployed

Create new test event

Test event

Save

Test

To invoke your function without saving an event, configure the JSON event, then choose Test.

Invocation type

☒ Synchronous

Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes.

☐ Asynchronous

Enqueues the Lambda function for execution and returns immediately with a request ID.

Event name

test

Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.

Event sharing settings

☒ Private

This event is only available in the Lambda console and to the event creator.

☐ Shareable

This event is available to IAM users within the same account who have permissions.

Info

Choose

Learn how cases in AI

Create

In this t to:

Build cons func that

Invoc its fu

Learn m

Star

Your Authtoken

Authenticates ngrok agents, SDKs, and the Kubernetes Operator. Keep it secret, like a password. Add it to the ngrok agent using any of the methods below.

.....



Copy

Building an integration or shared automation? Create a [Service User](#) and generate dedicated authtokens.

Attach policy to ecommerce-ingestion-lambda-role-5rrd7jc9

► Current permissions policies (1)

Other permissions policies (1/1222)

Filter by Type			1 match
Q AmazonS3FullAcces	All types		
Policy name	Type	Description	
AmazonS3FullAccess	AWS managed	Provides full access to all buckets via t...	
			Cancel Add permissions

604
605
606
607
608
609
610

```
"message":  
  "Failed to process orders",  
  
"error":  
  str(e),  
  
"error_type":
```

Asynchronous
Enqueues the Lambda function for execution and returns immediately with a request ID.

Event name
test

PROBLEMS
Status: Succeeded
Test Event Name: test

OUTPUT
Response:
{
 "statusCode": 200,
 "body": "{\"message\": \"Orders processed successfully\", \"orders_received\": 10, \"orders_uploaded\": 10, \"previous_order_id\": 2000000, \"last_order_id\": 2000010}"

CODE
REFERENCE
LOG
TERMINAL

Execution Results

aws-e-commerce-data-shereen-2026 [Info](#)[Objects](#) | [Metadata](#) | [Properties](#) | [Permissions](#) | [Metrics](#) | [Management](#) | [File systems](#) | [Access Points](#)

Objects (2)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 Inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

☐	Name	Type	Last modified	Size	Storage class
☐	api_raw/	Folder	-	-	-
☐	raw/	Folder	-	-	-

✔ **Successfully started job**
Successfully started job Task29. Navigate to [Run details](#) for more details.

Task29

Last modified on 9/22/2026, 8:45:03 PM

[Stop notebook](#)[Download Notebook](#)[Notebook](#) | [Script](#) | [Job details](#) | [Runs](#) | [Data quality](#) | [Schedules](#) | [Version Control](#)

```
shippers = read_csv("shippers.csv")
shipments = read_csv("shipments.csv")

=====
READING HISTORICAL DATA
=====
Reading: s3://aws-e-commerce-data-shereen-2026/raw/ecommerce/customers.csv
Reading: s3://aws-e-commerce-data-shereen-2026/raw/ecommerce/categories.csv
Reading: s3://aws-e-commerce-data-shereen-2026/raw/ecommerce/products.csv
Reading: s3://aws-e-commerce-data-shereen-2026/raw/ecommerce/departments.csv
Reading: s3://aws-e-commerce-data-shereen-2026/raw/ecommerce/employees.csv
Reading: s3://aws-e-commerce-data-shereen-2026/raw/ecommerce/suppliers.csv
Reading: s3://aws-e-commerce-data-shereen-2026/raw/ecommerce/orders.csv
Reading: s3://aws-e-commerce-data-shereen-2026/raw/ecommerce/order_details.csv
Reading: s3://aws-e-commerce-data-shereen-2026/raw/ecommerce/payments.csv
Reading: s3://aws-e-commerce-data-shereen-2026/raw/ecommerce/product_suppliers.csv
Reading: s3://aws-e-commerce-data-shereen-2026/raw/ecommerce/shippers.csv
Reading: s3://aws-e-commerce-data-shereen-2026/raw/ecommerce/shipments.csv
```

(All outputs appears in the uploaded notebook)

Task29

Last modified on 9/22/2026, 8:45:03 PM

[Stop notebook](#)

Notebook

Script

Job details

Runs

Data quality

Schedules

Version Control



Code

```
api_json.printSchema()
```

```
=====
READING API JSON
=====
```

```
API JSON schema:
```

```
root
```

```
 |-- metadata: struct (nullable = true)
 |   |-- event_id: string (nullable = true)
 |   |-- event_timestamp: string (nullable = true)
 |   |-- event_type: string (nullable = true)
 |   |-- source: string (nullable = true)
 |-- order: struct (nullable = true)
 |   |-- CustomerID: long (nullable = true)
 |   |-- OrderDate: string (nullable = true)
 |   |-- OrderID: long (nullable = true)
 |   |-- Status: string (nullable = true)
 |-- order_details: array (nullable = true)
 |   |-- element: struct (containsNull = true)
```

0

\$

1



Initialized (additional servers needed)

Glue PySpark | Idle



CodeWhisperer

```
s3 = boto3.client("s3")

response = s3.list_objects_v2(
    Bucket="aws-ecommerce-data-shereen-2026",
    Prefix="api_raw/orders/"
)

for obj in response.get("Contents", []):
    print(obj["Key"])
```

```
api_raw/orders/order_2000001.json
api_raw/orders/order_2000002.json
api_raw/orders/order_2000003.json
api_raw/orders/order_2000004.json
api_raw/orders/order_2000005.json
api_raw/orders/order_2000006.json
api_raw/orders/order_2000007.json
api_raw/orders/order_2000008.json
api_raw/orders/order_2000009.json
```

```
|    |-- total_amount: double (nullable = true)
```

```
[*]: from pyspark.sql import functions as F

BUCKET = "aws-ecommerce-data-shereen-2026"

API_RAW_PATH = f"s3://{BUCKET}/api_raw/orders/"

api_orders = (
    spark.read
    .option("multiLine", True)
    .json(API_RAW_PATH)
)

print("API records:", api_orders.count())

api_orders.printSchema()
```

```
api_orders.printSchema()
```

API records: 10

```
root
|-- metadata: struct (nullable = true)
|   |-- event_id: string (nullable = true)
|   |-- event_timestamp: string (nullable = true)
|   |-- event_type: string (nullable = true)
|   |-- source: string (nullable = true)
|-- order: struct (nullable = true)
|   |-- CustomerID: long (nullable = true)
|   |-- OrderDate: string (nullable = true)
|   |-- OrderID: long (nullable = true)
|   |-- Status: string (nullable = true)
|-- order_details: array (nullable = true)
|   |-- element: struct (containsNull = true)
|       |-- Discount: double (nullable = true)
```

🔗 Initialized (additional servers needed) Glue PySpark Idle ✓ CodeWhisperer

```

F.col("order.OrderID")
  .cast("long")
  .alias("orderid"),

F.col("order.CustomerID")
  .cast("long")
  .alias("customerid"),

F.to_date(
  F.col("order.OrderDate")
).alias("orderdate"),

F.upper(
  F.trim(
    F.col("order.Status")
  )
).alias("status"),

# API metadata
F.col("metadata.event_id")
  .alias("event_id"),

F.to_timestamp(
  F.col("metadata.event_timestamp")
).alias("event_timestamp"),

F.col("metadata.event_type")
  .alias("event_type"),

F.col("metadata.source")
  .alias("source")
)

api_orders_df.show(10, truncate=False)

```

orderid	customerid	orderdate	status	event_id	event_timestamp	event_type	source
2000009	20885	2026-09-22	CONFIRMED	f2cdbc11-f1b0-4c4d-ae75-772e1ac1171e	2026-09-22 15:06:56.670277	ORDER_CREATED	ordering_api
2000004	286647	2026-09-22	CONFIRMED	72e66691-97ce-42ce-8634-3199512dea57	2026-09-22 15:06:56.613488	ORDER_CREATED	ordering_api
2000001	366336	2026-09-22	CONFIRMED	48526c66-c11b-4bbd-bf0f-b369fdb5e2c4	2026-09-22 15:06:56.595536	ORDER_CREATED	ordering_api

```
[14]: api_order_details_df = (
    api_orders
    .select(

        F.col("order.OrderID")
        .cast("long")
        .alias("orderid"),

        F.explode(
            F.col("order_details")
        ).alias("detail")
    )
)
api_order_details_df.show()
```

```
+-----+-----+
|orderid|      detail|
+-----+-----+
|2000009|[15.0, 5000018, 2...|
|2000009|[15.0, 5000019, 2...|
|2000009|[0.0, 5000020, 20...|
|2000009|[0.0, 5000021, 20...|
|2000004|[10.0, 5000007, 2...|
|2000004|[5.0, 5000008, 20...|
|2000004|[0.0, 5000009, 20...|
|2000004|[15.0, 5000010, 2...|
|2000001|[15.0, 5000001, 2...|
|2000001|[0.0, 5000002, 20...|
|2000001|[15.0, 5000003, 2...|
|2000007|[0.0, 5000013, 20...|
|2000007|[15.0, 5000014, 2...|
|2000007|[0.0, 5000015, 20...|
|2000010|[10.0, 5000022, 2...|
|2000010|[15.0, 5000023, 2...|
|2000002|[0.0, 5000004, 20...|
|2000002|[5.0, 5000005, 20...|
|2000008|[5.0, 5000016, 20...|
|2000008|[0.0, 5000017, 20...|
```

```
[45]: all_orders_details.show(2)
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+
|orderdetailid|orderid|productid|quantity|unitprice|discount|gross_amount|discount_amount|total_amount|
+-----+-----+-----+-----+-----+-----+-----+-----+
|          28|  21929|      412|        8|  1768.21|    0.00|   14145.68|    0.000000|  14145.680000|
|          29|  27162|      707|        3|   521.53|    0.00|   1564.59|    0.000000|   1564.590000|
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 2 rows
```

```
[131]: all_orders_details.count()
```

```
5000274
```

```
[ ]:
```

```
[84]: write_parquet(dim_customer, "dim_customer")
      write_parquet(dim_product, "dim_product")
      write_parquet(dim_category, "dim_category")
      write_parquet(dim_department, "dim_department")
      write_parquet(dim_supplier, "dim_supplier")
      write_parquet(dim_employee, "dim_employee")
      write_parquet(dim_shipper, "dim_shipper")
      write_parquet(dim_date, "dim_date")
      write_parquet(dim_payment_method, "dim_payment_method")
      write_parquet(dim_order_status, "dim_order_status")

Writing dim_customer -> s3://aws-e-commerce-data-shereen-2026/curated/ecommerce/dim_customer
Writing dim_product -> s3://aws-e-commerce-data-shereen-2026/curated/ecommerce/dim_product
Writing dim_category -> s3://aws-e-commerce-data-shereen-2026/curated/ecommerce/dim_category
Writing dim_department -> s3://aws-e-commerce-data-shereen-2026/curated/ecommerce/dim_department
Writing dim_supplier -> s3://aws-e-commerce-data-shereen-2026/curated/ecommerce/dim_supplier
Writing dim_employee -> s3://aws-e-commerce-data-shereen-2026/curated/ecommerce/dim_employee
Writing dim_shipper -> s3://aws-e-commerce-data-shereen-2026/curated/ecommerce/dim_shipper
Writing dim_date -> s3://aws-e-commerce-data-shereen-2026/curated/ecommerce/dim_date
Writing dim_payment_method -> s3://aws-e-commerce-data-shereen-2026/curated/ecommerce/dim_payment_method
Writing dim_order_status -> s3://aws-e-commerce-data-shereen-2026/curated/ecommerce/dim_order_status
```

```
[81]: write_parquet(fact_order, "fact_order")
      write_parquet(fact_order_detail, "fact_order_detail")
      write_parquet(fact_payment, "fact_payment")
      write_parquet(fact_shipment, "fact_shipment")
      write_parquet(fact_customer_sales, "fact_customer_sales")
      write_parquet(fact_product_sales, "fact_product_sales")

Writing fact_order -> s3://aws-e-commerce-data-shereen-2026/curated/ecommerce/fact_order
Writing fact_order_detail -> s3://aws-e-commerce-data-shereen-2026/curated/ecommerce/fact_order_detail
Writing fact_payment -> s3://aws-e-commerce-data-shereen-2026/curated/ecommerce/fact_payment
Writing fact_shipment -> s3://aws-e-commerce-data-shereen-2026/curated/ecommerce/fact_shipment
Writing fact_customer_sales -> s3://aws-e-commerce-data-shereen-2026/curated/ecommerce/fact_customer_sales
Writing fact_product_sales -> s3://aws-e-commerce-data-shereen-2026/curated/ecommerce/fact_product_sales
```

```
1  Initialized (additional servers needed)  Glue PySpark | Idle  ✓ CodeWhisperer
```

```
Writing fact_product_sales -> s3://aws-e-commerce-data-shereen-2026/curated/ecommerce/fact_product_sales
```

```
85]: write_parquet(bridge_product_supplier, "bridge_product_supplier")
```

```
Writing bridge_product_supplier -> s3://aws-e-commerce-data-shereen-2026/curated/ecommerce/bridge_product_supplier
```

```
[ ]:
```

then we can find that curated folder automatically added

Objects (3)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

☐

Name

▲

Type

▼

Last modified

▼

Size

▼

Storage class

▼

☐	api_raw/	Folder	-	-	-	-
☐	curated/	Folder	-	-	-	-
☐	raw/	Folder	-	-	-	-

ecommerce/

Objects

Properties

Objects (18)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly

☐

Name

▲

Type

▼

Last modified

▼

Size

▼

☐	api_orders/	Folder	-	-	-
☐	bridge_product_supplier/	Folder	-	-	-
☐	dim_category/	Folder	-	-	-
☐	dim_customer/	Folder	-	-	-
☐	dim_date/	Folder	-	-	-
☐	dim_department/	Folder	-	-	-
☐	dim_employee/	Folder	-	-	-
☐	dim_order_status/	Folder	-	-	-
☐	dim_payment_method/	Folder	-	-	-
☐	dim_product/	Folder	-	-	-
☐	dim_shipper/	Folder	-	-	-
☐	dim_supplier/	Folder	-	-	-
☐	fact_customer_sales/	Folder	-	-	-
☐	fact_order_detail/	Folder	-	-	-
☐	fact_order/	Folder	-	-	-
☐	fact_payment/	Folder	-	-	-
☐	fact_product_sales/	Folder	-	-	-
☐	fact_shipment/	Folder	-	-	-

fact_customer_sales/

Copy S3 URI

Objects Properties

Objects (4)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	part-00000-133a7f8d-4b30-4c5d-abf8-5b02e673aab7-c000.snappy.parquet	parquet	September 22, 2026, 21:36:14 (UTC+03:00)	87.0 KB	Standard
☐	part-00001-133a7f8d-4b30-4c5d-abf8-5b02e673aab7-c000.snappy.parquet	parquet	September 22, 2026, 21:36:14 (UTC+03:00)	85.9 KB	Standard
☐	part-00002-133a7f8d-4b30-4c5d-abf8-5b02e673aab7-c000.snappy.parquet	parquet	September 22, 2026, 21:36:14 (UTC+03:00)	83.6 KB	Standard
☐	part-00003-133a7f8d-4b30-4c5d-abf8-5b02e673aab7-c000.snappy.parquet	parquet	September 22, 2026, 21:36:14 (UTC+03:00)	84.9 KB	Standard

AWS Glue

Getting started

SageMaker Unified Studio

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Data Catalog tables

Data connections

Workflows (orchestration)

Zero-ETL integrations

Data Catalog

Catalogs

Databases

Tables

Stream schema registries

Schemas

Connections

Crawlers

Classifiers

Catalog settings

Add crawler

Step 1

Set crawler properties

Step 2

Choose data sources and classifiers

Step 3

Configure security settings

Step 4

Set output and scheduling

Step 5

Review and create

Choose data sources and classifiers

Data source configuration

Is your data already mapped to Glue tables?

Not yet

Select one or more data sources to be crawled.

Yes

Select existing tables from your Glue Data Catalog.

Data sources (1)

The list of data sources to be scanned by the crawler.

Type

Data source

S3

s3://aws-ecommerce-data-shereen-2026/curated/

Custom classifiers - optional

A classifier checks whether a given file is in a format the crawler can handle. If it is, the classifier creates a schema in the form of a StructType object that matches that data format.

Cancel

Previous

Next

Add crawler

Step 1

Set crawler properties

Step 2

Choose data sources and classifiers

Step 3

Configure security settings

Step 4

Set output and scheduling

Step 5

Review and create

Configure security settings

IAM role

Existing IAM role

AWSGlueServiceRole-Default

Create new IAM role

Update chosen IAM role

Only IAM roles created by the AWS Glue console and have the prefix "AWSGlueServiceRole-" can be updated.

Lake Formation configuration - optional

Allow the crawler to use Lake Formation credentials for crawling the data source.

Use Lake Formation credentials for crawling S3 data source

Checking this box will allow the crawler to use Lake Formation credentials for crawling the data source. If the data source is registered in another account, you must provide the registered account ID. Otherwise, the crawler will crawl only those data sources associated to the account. Only applicable to S3, Glue Catalog, Iceberg, and Hudi data sources.

Security configuration - optional

Enable at-rest encryption with a security configuration.

Cancel

Previous

Next

Create a database

Create a database in the AWS Glue Data Catalog.

Database type

- ☒ **Glue Database**
Create a standard Glue database in the Data Catalog.
- ☐ **Glue Database Resource Link**
Create a resource link to a database in another Data Catalog.
- ☐ **Glue Database in S3 Tables Federated Catalog**
Create a database in an S3 Tables federated catalog.

Database details

Name

Database name is required, in lowercase characters, and no longer than 255 characters.

Description - optional

Descriptions can be up to 2048 characters long.

Location - optional

Set the URI location for use by clients of the Data Catalog.

An S3 location is required for managed tables and Zero-ETL integrations.

One crawler successfully created
The following crawler is now created: "ecommerce-curated-crawler"

ecommerce-curated-crawler

Last updated (UTC)
September 22, 2026 at 18:56:53

Run crawler

Edit

Delete

Crawler properties

Name	ecommerce-curated-crawler	IAM role	AWSGlueServiceRole-Default	Database	ecommerce_curated_db	State	READY
Description	-	Security configuration	-	Lake Formation configuration	-	Table prefix	-
Maximum table threshold	-						

Advanced settings

Crawler runs

Schedule

Data sources

Classifiers

Tags

Advanced settings

Crawler runs | Schedule | Data sources | Classifiers | Tags

Crawler runs (1)

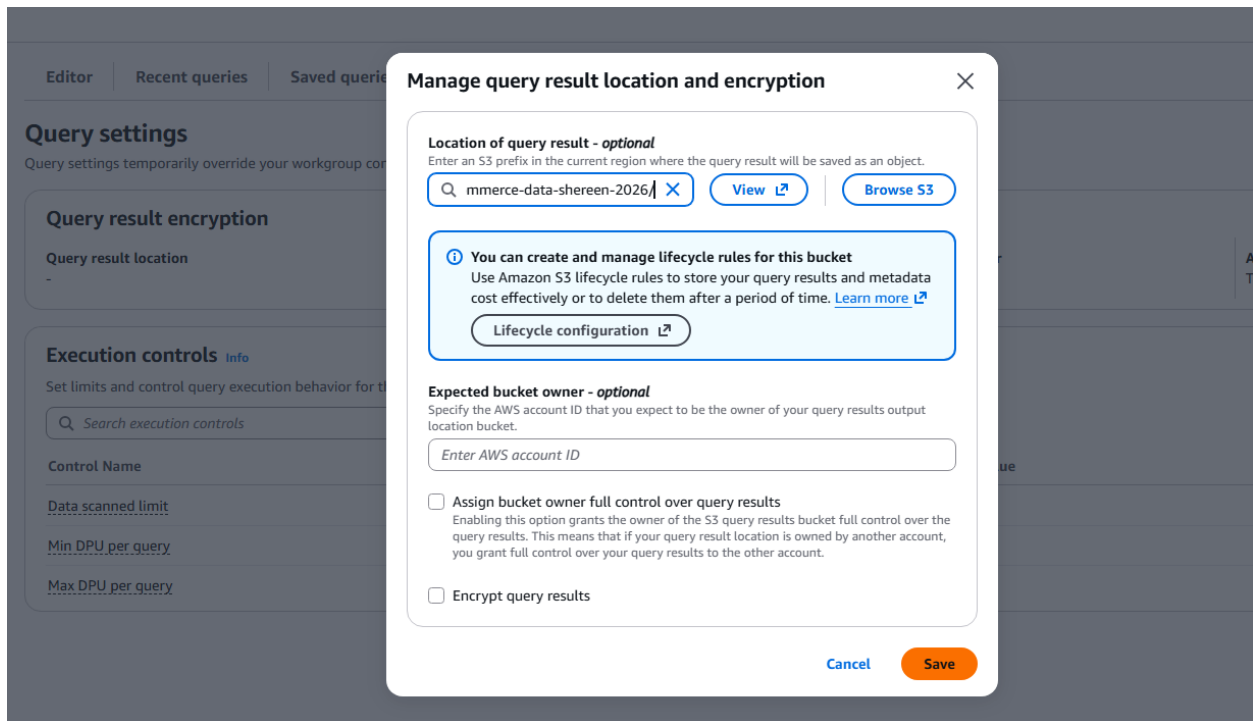
The list of crawler runs for this crawler.

Stop run

View CloudWatch logs

View run details

Start time (UTC)	End time (UTC)	Current/last duration	Status	DPU hours	Table changes
September 22, 2026 at 18:57:23	September 22, 2026 at 18:58:01	38 s	Completed	0.111	18 table changes, 1 partition change



Task 29: Overall Business Performance

The interface shows a SQL query editor on the left and a results pane on the right.

SQL Query:

```
1 SELECT
2   COUNT(DISTINCT customer_id) AS total_customers,
3   SUM(total_orders) AS total_orders,
4   SUM(total_quantity) AS total_quantity_sold,
5   SUM(total_sales) AS total_sales,
6   SUM(total_sales * 0.25) AS total_estimated_profit
7 FROM fact_customer_sales;
```

Query Stats: Time in queue: 94 ms, Run time: 710 ms, Data scanned: 112.39 KB.

Results (1):

#	total_customers	total_orders	total_quantity_sold	total_sales	total_estimated_profit
1	9869	43146	549538	781063824.39	1.952659560974999E8

Task 30: Monthly Aggregations

1 SELECT
2 DATE_TRUNC('month', first_order_date) AS order_month,
3 SUM(total_orders) AS total_orders,
4 SUM(total_quantity) AS total_quantity,
5 SUM(total_sales) AS total_sales
6 FROM fact_customer_sales
7 GROUP BY 1
8 ORDER BY order_month;

SQL Ln 8, Col 22

Run again

Explain

Cancel

Clear

Create

Reuse query results

up to 60 minutes ago

Query results

Query stats

Completed

Time in queue: 101 ms

Run time: 569 ms

Data scanned: 95.85 KB

Results (33)

Copy

Download results CSV

Search rows

< 1 >

#	order_month	total_orders	total_quantity	total_sales
1	2024-01-01	6828	87465	124902231.96
2	2024-02-01	5161	65186	92049533.40

Task 31: Sales by Customer Segment / Details

Query 1

```
1 SELECT
2   customer_segment,
3   COUNT(customer_id) AS customer_count,
4   SUM(total_orders) AS total_orders,
5   SUM(total_sales) AS total_sales
6 FROM fact_customer_sales
7 GROUP BY customer_segment
8 ORDER BY total_sales DESC;
```

SQLLn 8, Col 27

Run again

Explain

Cancel

Clear

Create

☐ Reuse query results
up to 60 minutes ago

Query results

Query stats

Completed

Time in queue: 83 msRun time: 664 msData scanned: 101.06 KB

Results (4)

CopyDownload results CSV

Search rows

#	customer_segment	customer_count	total_orders	total_sales
1	VIP	9575	42769	779608426.70
2	PREMIUM	143	204	1082996.42
3	REGULAR	121	143	356304.47
4	LOW_VALUE	30	30	16096.80

Task 32: Top 10 Products by Sales

✓ Query 1 ⋮

```

1 SELECT
2     product_id,
3     SUM(total_sales) AS total_sales,
4     SUM(units_sold) AS total_units_sold
5 FROM fact_product_sales
6 GROUP BY product_id
7 ORDER BY total_sales DESC
8 LIMIT 10;

```

SQL Ln 8, Col 10

Query results

Query stats

✓ Completed

Time in queue: 134 ms

Run time: 538 ms

Data scanned: 13.52 KB

Results (10)

Copy

Download results CSV

Search rows

< 1 > ⚙

#	product_id	total_sales	total_units_sold
1	582	1973379.08	714
2	794	1903432.08	711
3	650	1833759.72	661
4	71	1752721.19	655
5	911	1729415.16	662
6	125	1710948.42	636
7	191	1708157.61	632
8	232	1707010.36	637
9	212	1704521.63	618
10	335	1700456.24	609

Task 33: Top 10 Customers by Total Spending

✓ Query 1 ⋮

```
1 SELECT
2     customer_id,
3     customer_segment,
4     total_orders,
5     total_sales AS total_spending
6 FROM fact_customer_sales
7 ORDER BY total_spending DESC
8 LIMIT 10;
```

SQL Ln 8, Col 10

Query results

Query stats

✓ Completed

Time in queue: 64 ms

Run time: 589 ms

Data scanned: 101.06 KB

Results (10)

Copy

Download results CSV

Search rows

< 1 > ⚙

#	▼	customer_id	▼	customer_segment	▼	total_orders	▼	total_spending	▼
1		6772		VIP		12		340149.01	
2		9895		VIP		7		310614.46	
3		245		VIP		9		299887.18	
4		4971		VIP		11		296960.55	
5		5446		VIP		9		295520.71	
6		5688		VIP		11		289294.45	
7		8502		VIP		12		286360.70	
8		4231		VIP		9		285589.92	
9		7013		VIP		10		284946.41	
10		849		VIP		13		283591.18	

Task 34: Average Order Value (AOV)

1

SELECT

2

SUM(total_sales) / NULLIF(SUM(total_orders), 0) AS average_order_value

3

FROM fact_customer_sales;

SQLLn 3, Col 26

Run again

▼

Explain ↗

Cancel

Clear

Create ▼

Query results

Query stats

✔ Completed

Time in queue: 102 ms

Run time: 404 ms

Results (1)

Copy

🔍 Search rows

#

▼

average_order_value

1

18102.81

Task 35: Order Status Analysis

Query 1

```
1 SELECT
2     s.order_status,
3     COUNT(o.order_id) AS order_count
4 FROM fact_order o
5 JOIN dim_order_status s ON o.order_status_id = s.order_status_id
6 GROUP BY s.order_status
7 ORDER BY order_count DESC;
```

[Run again](#)[Explain](#)[Cancel](#)[Clear](#)[Create](#)

☒ Reuse query results
up to 60 minutes ago

[Query results](#)[Query stats](#)

Completed

Time in queue: 97 ms

Run time: 761 ms

Data scanned: 215.53 KB

Results (6)

[Copy](#)[Download results CSV](#)

< 1 > ⚙

#	order_status	order_count
1	PENDING	10044
2	SHIPPED	10015
3	CANCELLED	9998
4	DELIVERED	9975
5	PROCESSING	9968
6	CONFIRMED	10

Task 36: Payment Methods Analysis

Query 1

```
1 SELECT
2     payment_method_id,
3     COUNT(payment_id) AS transaction_count,
4     SUM(amount) AS total_payment_amount,
5     AVG(amount) AS average_payment_amount
6 FROM fact_payment
7 GROUP BY payment_method_id
8 ORDER BY total_payment_amount DESC;
```

SQL Ln 8, Col 36

Query results Query stats

Completed

Time in queue: 79 ms

Run time: 510 ms

Data scanned: 407.97 KB

Results (5)

Copy

Download results CSV

Search rows

< 1 > ⚙

#	payment_method_id	transaction_count	total_payment_amount	average_payment_amount
1	3	9091	23217087.74	2553.85
2	2	9082	23078562.34	2541.13
3	1	9027	22611260.23	2504.85
4	5	8929	22196526.05	2485.89
5	4	8881	22024153.51	2479.92

Task 37: Shipment Performance Analysis

1 SELECT
2 COUNT(shipment_id) AS total_shipments,
3 AVG(shipping_days) AS avg_delivery_days,
4 MIN(shipping_days) AS min_delivery_days,
5 MAX(shipping_days) AS max_delivery_days
6 FROM fact_shipment;

SQL Ln 6, Col 20

Run again Explain Cancel Clear Create Reuse query results up to 60 minutes ago

Query results Query stats

Completed Time in queue: 62 ms Run time: 540 ms Data scanned: 178.89 KB

Results (1) Copy Download results CSV

Search rows

#	total_shipments	avg_delivery_days	min_delivery_days	max_delivery_days
1	40010	5.493526618345413	1	10

Task 38: Running Sales (Cumulative Sales)

1 SELECT
2 first_order_date AS order_date,
3 SUM(total_sales) AS daily_sales,
4 SUM(SUM(total_sales)) OVER (ORDER BY first_order_date) AS running_total_sales
5 FROM fact_customer_sales
6 GROUP BY first_order_date
7 ORDER BY order_date;

SQL Ln 7, Col 21

Run again Explain Cancel Clear Create Reuse query results up to 60 minutes ago

Query results Query stats

Completed Time in queue: 101 ms Run time: 589 ms Data scanned: 78.06 KB

Results (874) Copy Download results CSV

Search rows

#	order_date	daily_sales	running_total_sales
1	2024-01-01	3628661.37	3628661.37
2	2024-01-02	3307827.87	6936489.24

Task 39: Month-over-Month (MoM) Growth

Query 1

```

4      SUM(total_sales) AS current_month_sales
5  FROM fact_customer_sales
6  GROUP BY 1
7  )
8  SELECT
9      order_month,
10     current_month_sales,
11     LAG(current_month_sales, 1) OVER (ORDER BY order_month) AS previous_month_sales,
12     current_month_sales - LAG(current_month_sales, 1) OVER (ORDER BY order_month) AS sales_difference,
13     ROUND(
14         (current_month_sales - LAG(current_month_sales, 1) OVER (ORDER BY order_month))
15         * 100.0 / NULLIF(LAG(current_month_sales, 1) OVER (ORDER BY order_month), 0), 2
16     ) AS growth_percentage
17 FROM monthly_sales
18 ORDER BY order_month;

```

SQL Ln 18, Col 22

Query results

Query stats

Completed

Time in queue: 89 ms

Run time: 501 ms

Data scanned: 78.06 KB

Results (33)

Copy

Download results CSV

Search rows

#	order_month	current_month_sales	previous_month_sales	sales_difference	growth_percentage
1	2024-01-01	124902231.96			
2	2024-02-01	92049533.40	124902231.96	-32852698.56	-26.3
3	2024-03-01	90148867.46	92049533.40	-1900665.94	-2.06
4	2024-04-01	74487795.34	90148867.46	-15661072.12	-17.37
5	2024-05-01	64710260.59	74487795.34	-9777534.75	-13.13
6	2024-06-01	52535905.88	64710260.59	-12174354.71	-18.81
7	2024-07-01	45210961.17	52535905.88	-7324944.71	-13.94
8	2024-08-01	37856437.37	45210961.17	-7354523.80	-16.27
9	2024-09-01	32373449.42	37856437.37	-5482987.95	-14.48
10	2024-10-01	26039883.91	32373449.42	-6333565.51	-19.56
11	2024-11-01	22134968.87	26039883.91	-3904915.04	-15.0

Task 41: Customer Ranking by Spending

Query 1 :

```
1 SELECT
2     customer_id,
3     total_sales AS total_spending,
4     total_orders,
5     RANK() OVER (ORDER BY total_sales DESC) AS rank
6 FROM fact_customer_sales
7 ORDER BY rank;
```

Query results

Query stats

Completed

Time in queue: 118 ms

Run time: 523 ms

Data scanned: 99.91 KB

Results (9,869)

Copy

Download results CSV

Search rows

#	customer_id	total_spending	total_orders	rank
1	6772	340149.01	12	1
2	9895	310614.46	7	2
3	245	299887.18	9	3
4	4971	296960.55	11	4
5	5446	295520.71	9	5
6	5688	289294.45	11	6
7	8502	286360.70	12	7
8	4231	285589.92	9	8

Part 14 — Final Architecture

